



**DRISHTI**  
*A Revolutionary Concept*

# MakerNova 2.0



**OBJECT  
FOLLOWING BOT**

### **MENTORS**

|                    |
|--------------------|
| 1. Mihir Kumar Roy |
| 2. Krutika Joshi   |
| 3. Rishika Agarwal |

### **TEAM MEMBERS**

|                       |
|-----------------------|
| 1. Aditya Kumbhar     |
| 2. Aman Mourya        |
| 3. Arya Shah          |
| 4. Avanish Gupta      |
| 5. Prakhar Jain       |
| 6. Shrutika Ingole    |
| 7. Snigdha Chatterjee |
| 8. Sri Harshitha B    |
| 9. Tanishq Rathore    |
| 10. Vidhi Mamanian    |

## **ACKNOWLEDGEMENT**

We are deeply thankful to our academic mentors for their unconditional advice, useful criticism and support to put this project together. Their assistance has been contributory in widening our knowledge on localization techniques and robotic navigation, and they have our endless appreciation on this. Our many thanks go to the Drishti Lab at SVNIT for their provision of the equipment, resources and technical assistance necessary for the completion of this project. Finally, we commend the researchers and authors who have written on this topic and who have guided us in this project because of their related works contributed to this specific project. Thank you all for your invaluable support and inspiration.

## **ABSTRACT**

This project presents the development of an autonomous robot capable of real-time object detection, tracking, and following, utilizing computer vision and deep learning techniques. Leveraging the YOLOv8 (You Only Look Once) object detection model, integrated with OpenCV, the robot identifies and tracks specific objects with high accuracy. The processing is handled by the Nvidia GTX 4050 GPU from a laptop, while an Arduino controls the movement of the bot communicating with the laptop through Pyserial, facilitating precise object following. The custom YOLOv8 model was trained on a dataset of target objects to enhance detection accuracy. This project aims to learn object detection, software and hardware implementation and contribute to the advancement of autonomous robotics.

# INDEX

|                                                                                           |           |
|-------------------------------------------------------------------------------------------|-----------|
| <b>ACKNOWLEDGEMENT.....</b>                                                               | <b>3</b>  |
| <b>ABSTRACT.....</b>                                                                      | <b>4</b>  |
| <b>INDEX.....</b>                                                                         | <b>5</b>  |
| <b>1.1 PROBLEM STATEMENT.....</b>                                                         | <b>6</b>  |
| <b>1.2 INTRODUCTION.....</b>                                                              | <b>6</b>  |
| <b>2. SOFTWARE APPLICATIONS FOR SIMULATION AND CODE DEPLOYMENT.....</b>                   | <b>7</b>  |
| 2.1 Arduino IDE.....                                                                      | 7         |
| 2.2 Python.....                                                                           | 7         |
| 2.3 Wokwi Simulator.....                                                                  | 7         |
| 2.4 Google Colaboratory.....                                                              | 7         |
| <b>3. PYTHON LIBRARIES.....</b>                                                           | <b>8</b>  |
| 3.1 YOLOv10.....                                                                          | 8         |
| 3.2 NumPy.....                                                                            | 8         |
| 3.3 OpenCV.....                                                                           | 8         |
| 3.4 PySerial.....                                                                         | 8         |
| 3.6 Cuda.....                                                                             | 9         |
| 3.7 Supervision.....                                                                      | 9         |
| <b>4. COMPONENTS.....</b>                                                                 | <b>10</b> |
| 4.1 ARDUINO.....                                                                          | 11        |
| 4.2 ARDUINO MEGA 2560.....                                                                | 11        |
| 4.3 MOTOR DRIVER (RHINO MDD20A).....                                                      | 12        |
| 4.4 ULTRASONIC SENSOR.....                                                                | 15        |
| 4.5 BUCK CONVERTER.....                                                                   | 17        |
| <b>5. COMMUNICATION PROTOCOLS.....</b>                                                    | <b>19</b> |
| 5.1 UART (Universal Asynchronous Receiver/Transmitter).....                               | 19        |
| 5.2 SPI (Serial Peripheral Interface).....                                                | 20        |
| 5.3 I2C (Inter-Integrated Circuit).....                                                   | 21        |
| <b>6. YOLO (You Only Look Once).....</b>                                                  | <b>24</b> |
| 6.1 Training A Model Using Google Colab.....                                              | 25        |
| 6.2 Algorithm for Object Detection and Motor Control Using YOLO,OpenCV, and PySerial..... | 27        |
| <b>7. CODE FOR OBJECT DETECTION.....</b>                                                  | <b>30</b> |
| <b>8. CODE FOR ARDUINO AND MOTOR CONTROL.....</b>                                         | <b>33</b> |
| <b>9. PYSERIAL.....</b>                                                                   | <b>35</b> |
| <b>10. CIRCUIT DIAGRAM:.....</b>                                                          | <b>38</b> |
| <b>11. ERRORS AND THEIR SOLUTIONS.....</b>                                                | <b>39</b> |
| <b>12. REFERENCES.....</b>                                                                | <b>41</b> |

## **1.1 PROBLEM STATEMENT**

An autonomous bot which can identify, track and follow a specific object in real time, navigating dynamic environments with minimal human intervention.

## **1.2 INTRODUCTION**

An object-following bot is a kind of autonomous robot capable of tracking, and following a specified object in real-time. The bot, using sensors, computer vision and YOLO algorithms, and motor control systems, adjust for the object it is targeting and change its path as the object moves.

This project uses high-level computer vision with advanced, real-time processing. The robot is supposed to detect, track, and follow a target object by using a camera with YOLO algorithm on the live feed through the camera using the OpenCV library of python. The processing part is planned to be performed by a powerful NVidia RTX 4050 GPU running through a laptop mounted on the bot, while the movement was controlled by a microcontroller with Arduino Mega. Together, these technologies will enable the bot to autonomously navigate dynamic environments, continually adjusting its movements to stay focused on the target object, making it an effective solution for real-time object tracking and following tasks.

## **2. SOFTWARE APPLICATIONS FOR SIMULATION AND CODE DEPLOYMENT**

### **2.1 Arduino IDE**

The Arduino IDE is a user-friendly integrated development environment designed for programming Arduino microcontrollers. It allows developers to write, compile, and upload code in a simplified version of C/C++ directly to Arduino boards. Widely used in embedded systems, robotics, the Arduino IDE supports various hardware components like sensors, motors, and LEDs, making it accessible for both beginners and professionals to prototype and deploy hardware solutions efficiently.

### **2.2 Python**

Python plays a central role in developing the bot's real-time object detection and tracking system. Its simplicity and extensive library ecosystem make it ideal for handling various tasks. Key libraries include OpenCV for computer vision, Ultralytics for implementing the YOLO algorithm, and Torch for deep learning model integration. NumPy is used for efficient numerical computations, while PySerial manages serial communication between the laptop and the Arduino Mega. Additionally, CUDA for GPU acceleration, enhancing real-time performance. These libraries, along with Python's versatility and readability, enable seamless development and deployment of the algorithms and data processing ensuring effective control and tracking in dynamic environments.

### **2.3 Wokwi Simulator**

Wokwi Simulator is an online platform that enables users to simulate microcontroller projects, like those involving Arduino and ESP32, without physical hardware. It's a valuable tool for prototyping and debugging circuits and code, allowing quick iterations and refinement before actual deployment.

### **2.4 Google Colaboratory**

Google Colaboratory, or Colab, is a cloud-based service that offers a Jupyter notebook environment for writing and executing Python code. It is particularly useful for training custom YOLO datasets, as it provides free access to powerful GPUs and TPUs, enabling efficient processing of computationally intensive tasks. Colab is also ideal for collaborative

## **3. PYTHON LIBRARIES**

### **3.1 YOLOv10**

YOLOv10, the latest version in the "You Only Look Once" series, enhances real-time object detection with improved speed and accuracy. Building on the strengths of YOLOv8, it excels in applications such as surveillance, robotics, and autonomous vehicles. Its advanced capabilities enable rapid and precise identification and tracking of objects in images and videos, making it an essential tool for scenarios requiring real-time analysis.

### **3.2 NumPy**

NumPy is a fundamental library for numerical computing in Python, providing support for large, multi-dimensional arrays and matrices. It also includes a comprehensive collection of mathematical functions that are essential for scientific computing. NumPy serves as the backbone for many other libraries in the Python ecosystem, such as Pandas for data manipulation and TensorFlow for machine learning. Its efficiency and flexibility make it a cornerstone of Python's capabilities in handling large-scale numerical data.

### **3.3 OpenCV**

OpenCV (Open Source Computer Vision Library) is an open-source library designed for computer vision and image processing tasks. It provides a wide range of tools for operations such as object detection, facial recognition, motion tracking, and image transformation. OpenCV is extensively used in robotics, security systems, and other applications where analyzing visual data is critical. Its robust functionality and widespread adoption make it a go-to solution for developers working on projects that require visual input processing.

### **3.4 PySerial**

This Python library facilitates serial data transfer, allowing the bot to read sensor data and control hardware components effectively. PySerial simplifies the process of interfacing with the Arduino, making it essential for managing real-time interactions and commands in robotics and embedded systems.

### **3.5 Ultralytics**

The YOLO algorithm performs object detection in real time, and for this task, the Ultralytics library plays a major role. Ultralytics provides an end-to-end clear interface for training and deployment with YOLO models, hence allowing their easy integration in Python for object

tracking and following. With such advanced functionality provided by YOLO through the Ultralytics library, the detection of target objects is going to be ensured in less time and more accurately in order to take care of the bot performance in dynamic environments.

### **3.6 Cuda**

CUDA is a programming model and parallel computing platform that helps harness the power of GPUs to speed up computing applications. CUDA libraries are a collection of pre-built functions that help developers leverage GPUs and focus on priority processes. It enables the user to access the computational resources of NVIDIA GPUs.

### **3.7 Supervision**

The Supervision library is a tool designed to simplify the process of post-processing outputs from computer vision models like YOLO, making it easier to handle bounding boxes, draw visualizations, and work with the outputs of various object detection algorithms. It helps in object identification, tracking, filtering, and its visualization

## 4. COMPONENTS

| Sr.No | COMPONENTS                  | DESCRIPTION                                                                                                                                                                                                                                                                           | QUANTITY |
|-------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1.    | Arduino Mega                | The Arduino Mega is a microcontroller board based on the ATmega2560 chip. It features numerous digital and analog I/O pins, substantial memory, and multiple communication interfaces, making it ideal for complex projects requiring extensive connectivity and computational power. | 1        |
| 2.    | Motor Driver (Rhino MDD20A) | The Rhino MDD20A is a motor driver designed for controlling two DC motors with up to 20A continuous current per channel. It is built for high-power applications in robotics and automation, with built-in protection features such as overcurrent and thermal shutdown               | 1        |
| 3.    | General Circuit Board (GCB) | A GCB is a flat board that supports and connects electronic components through soldered conductive pathways, providing a structured layout for electronic circuits.                                                                                                                   | 1        |
| 4.    | Omni-Wheels                 | Omni-wheels are designed for multidirectional movement by using multiple rollers around the wheel's edge. This allows for smooth and versatile maneuvering, ideal for precise and agile movement                                                                                      | 1        |
| 5.    | LiPo Battery(12V)           | A LiPo (Lithium Polymer) battery is a lightweight, rechargeable power source known for its high energy density and flexible shape. It is commonly used in devices such as drones and RC vehicles due to its compact and powerful design.                                              | 1        |
| 6.    | Lenovo 300 FHD Webcam       | The Webcam provides real-time image capture, essential for tasks like object detection and tracking in the bot, ensuring clear video input.                                                                                                                                           | 1        |
| 7.    | Pin Headers                 | These are used to connect wires, sensors, or other peripherals (like the camera, motor driver) to Arduino, making it easier to connect hardware components without needing to solder wires directly, allowing for easy assembly, modification, or replacement of parts.               | 10       |

## 4.1 ARDUINO

Arduino is an Italian open-source hardware and software company, project, and user community that designs and manufactures single-board microcontrollers and microcontroller kits for building digital devices.

## 4.2 ARDUINO MEGA 2560

The Arduino Mega is a powerful microcontroller board designed for projects that require more I/O pins and memory than the Arduino Uno. It's also an open-source electronics platform, offering flexibility and ease of use.

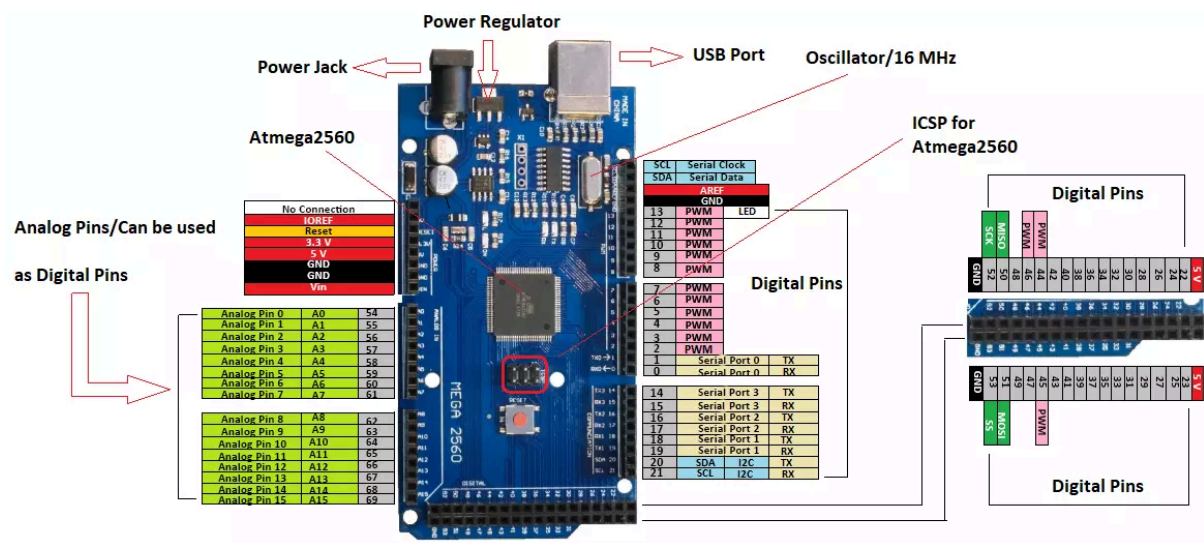


Figure 1 : Arduino Mega 2560

## Technical Specifications

- **Microcontroller:** ATmega2560
- **Operating Voltage:** 5V
- **Input Voltage (recommended):** 7-12V
- **Input Voltage (limits):** 6-20V
- **Digital I/O Pins:** 54 (of which 15 can provide PWM output)
- **Analog Input Pins:** 16
- **DC Current per I/O Pin:** 40 mA
- **DC Current for 3.3V Pin:** 50 mA
- **Clock Speed:** 16 MHz
- **LED\_BUILTIN:** Pin 13

## Key Components

- **ATmega2560 Microcontroller:** The central processing unit, handling input and output tasks.
- **Digital I/O Pins:** 54 pins for reading and writing digital signals. Pins 0 and 1 are used for serial communication.
- **Analog Input Pins:** 16 pins (A0 to A15) for reading analog signals from sensors.
- **Power Jack:** For external power supply (7-12V recommended).
- **USB Connector:** Used for programming and powering the board via USB.
- **Reset Button:** Resets the microcontroller.
- **Power Pins:** Includes 5V, 3.3V, GND, and Vin pins for power management.

## Advantages and Key Features

- **Increased I/O Capabilities:** Offers more digital and analog pins, additional PWM outputs, and enhanced I/O options, making it suitable for larger and more complex projects.
- **Greater Memory Capacity:** Provides expanded flash memory, SRAM, and EEPROM storage for more extensive programs, data, and complex code.
- **Compatibility and Community Support:** Compatible with various shields and modules, boosting functionality and project possibilities, with robust community support and numerous resources, tutorials, and libraries available.

## Limitations

- **Size:** Larger size compared to the Arduino Uno, which may be a consideration for projects with size constraints.
- **Power Consumption:** Higher power consumption, which could impact battery-operated projects.
- **Complexity:** The increased number of pins and features might be overwhelming for beginners compared to simpler boards.

## 4.3 MOTOR DRIVER (RHINO MDD20A)

The Rhino Dual Driver 20A provides bidirectional control for two high-power brushed DC motors with a voltage range of 6V to 30V. It features a full discrete NMOS H-Bridge design that supports 20A continuously per motor without a heatsink. Unlike other motor drivers that

overstate current ratings based on MOSFET specs, Rhino's driver is tested in the lab under full load with a thermal camera to ensure accurate ratings.

The driver includes onboard test buttons and motor output LEDs for quick functional tests without a host controller. It is controlled via PWM and DIR inputs, and supports logic voltages from 1.8V to 12V, making it compatible with various controllers like Arduino and Raspberry Pi.

Protection features include overcurrent protection to prevent damage from motor stalls or oversized motors, and temperature-based current limiting, which adjusts the maximum current based on the board's temperature. All inputs are electrical and opto-isolated.

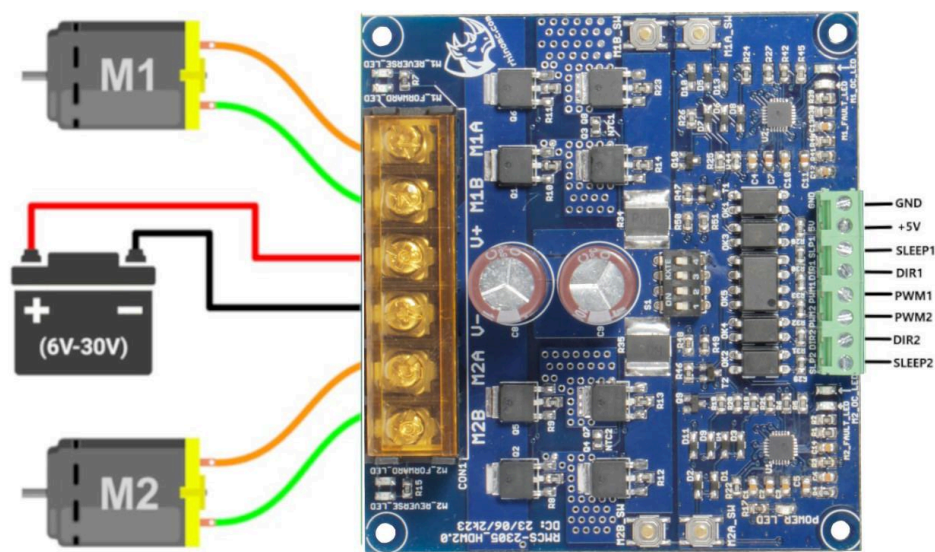


Figure 2: Rhino Motor Driver

## Overview

- Motor Channel: 2
- Operating Voltage (VDC): 6 to 30 VDC
- Peak Current (A): 60 (10 seconds)
- Continuous Current (A): 20
- Motor Output PWM Frequency : 20 kHz

## Features

- Bidirectional control for one brushed DC motor.
- Buttons for quick testing.
- Inputs compatible with 1.8V, 3.3V, 5V and 12 logic (Arduino, Raspberry Pi, PLC, etc).

- PWM and DIR inputs.
- LEDs for motor output state.
- PWM frequency up to 20kHz (Output frequency is the same as input frequency).
- Compatible with sign-magnitude and locked-antiphase PWM operation.
- Overcurrent protection with active current limiting.
- Temperature protection.
- Undervoltage shutdown.

## Understanding code

### *Pseudocode for Rhino MDD Motor Driver Control*

#### Define Pin Mappings:

- sleep1: Pin 2 (Motor M1 sleep control)
- Dir1: Pin 3 (Motor M1 direction control)
- pwm1: Pin 5 (Motor M1 PWM input)
- pwm2: Pin 6 (Motor M2 PWM input)
- Dir2: Pin 7 (Motor M2 direction control)
- sleep2: Pin 8 (Motor M2 sleep control)

#### Initialize Pins:

- Set sleep1, Dir1, pwm1, sleep2, Dir2, and pwm2 as output pins.

#### Cycle Operation:

##### 1. Forward Movement:

- Set PWM for both motors to 230 (speed).
- Activate motors by setting sleep1 and sleep2 to LOW.
- Set Dir1 and Dir2 to LOW for forward motion.
- Wait for 5 seconds.

##### 2. Stop Motors:

- Deactivate motors by setting sleep1 and sleep2 to HIGH.
- Wait for 2 seconds.

##### 3. Backward Movement:

- Activate motors by setting sleep1 and sleep2 to LOW.
- Set Dir1 and Dir2 to HIGH for reverse motion.
- Wait for 5 seconds.

##### 4. Stop Motors:

- Deactivate motors by setting `sleep1` and `sleep2` to `HIGH`.
- Wait for 2 seconds.

## 5. Repeat the Cycle.

## 4.4 ULTRASONIC SENSOR

The HC-SR04 ultrasonic sensor is utilized for precise distance measurement and obstacle detection enabling the bot to navigate around obstacles and avoid collisions in real-time.



Figure 3: Ultrasonic Sensor

## How the HC-SR04 Ultrasonic Sensor Works

### Transmitter:

- **Ultrasonic Pulse Generation:** The HC-SR04 sensor emits high-frequency sound waves, typically around 40 kHz, using a piezoelectric transducer. These sound waves are inaudible to humans and are generated in short bursts.
- **Pulse Characteristics:** The pulses are brief, usually around 40  $\mu$ s in duration, and are transmitted in bursts of eight. This approach enhances the accuracy of distance measurements and minimizes noise interference.

## Receiver:

- **Echo Detection:** The same piezoelectric transducer used for emitting the sound waves also functions as the receiver. When the emitted sound waves hit an object, they bounce back and are captured by the transducer.
- **Time Measurement:** The sensor calculates the time elapsed between the transmission of the sound pulse and the reception of the echo. This time interval is directly proportional to the distance between the sensor and the object.

## Key Features and Benefits:

- **Accuracy:** Provides precise distance measurements with high accuracy.
- **Range:** Capable of detecting objects within a broad range, from a few centimeters up to several meters.
- **Versatility:** Effective in a variety of environments and applications.
- **Reliability:** Durable and robust, suitable for demanding conditions.

## Specifications:

- **Operating Voltage:** 3.3V to 5V DC
- **Current Consumption:** Less than 2mA
- **Measurement Range:** 2 cm to 400 cm
- **Working Frequency:** 40 kHz
- **Trigger Pulse Width:** Minimum 10  $\mu$ s
- **Echo Pulse Width:** Proportional to the distance measured

## Pin Configuration:

- **Vcc:** Provides power to the sensor, typically requiring a voltage between 3.3V and 5V.
- **Trigger:** Initiates a measurement. Applying a high-level pulse of at least 10  $\mu$ s to this pin causes the sensor to emit eight ultrasonic pulses and start timing the echo's return.
- **Echo:** Outputs the measured distance. The length of the high-level pulse on this pin is directly proportional to the distance between the sensor and the object; a longer pulse indicates a greater distance.
- **Ground:** Connects the sensor to the ground, providing a reference point for the sensor's internal circuitry.

## Pseudo Code For HC-SR04

### Initialize:

- Define the trigPin as Pin 9 (Trigger pin).
- Define the echoPin as Pin 10 (Echo pin).
- Set up variables duration (to store the time taken for the pulse to return) and distance (to store the calculated distance).

### Setup:

- Begin serial communication at 9600 baud rate for monitoring.
- Configure trigPin as an output pin.
- Configure echoPin as an input pin.

### Main Loop:

- **Prepare Sensor:**
  - Set trigPin to LOW to clear any residual signals.
  - Wait for 2 microseconds.
- **Send Trigger Pulse:**
  - Set trigPin to HIGH for 10 microseconds to generate an ultrasonic burst.
  - Set trigPin back to LOW.
- **Measure Echo:**
  - Measure the duration of the pulse received at echoPin using pulseIn() function.
- **Calculate Distance:**
  - Calculate the distance using the formula:  $\text{distance} = (\text{duration}/2) \times 0.034$
  - This formula accounts for the speed of sound (34300 cm/s) and divides the duration by 2 since the signal travels to the object and back.
- **Display Result:**
  - Print the calculated distance to the serial monitor in centimeters.
- **Pause:**
  - Wait for 500 milliseconds before taking the next measurement to avoid excessive sampling.

## 4.5 BUCK CONVERTER

A buck converter is a type of DC-DC power converter that steps down voltage from a higher level to a lower level while increasing current. It's commonly used to power electronic devices from a higher voltage source efficiently.

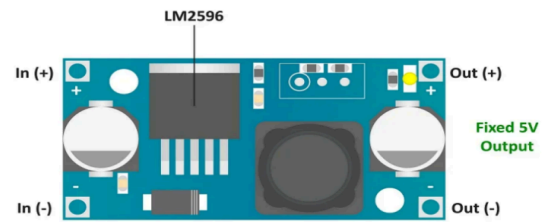
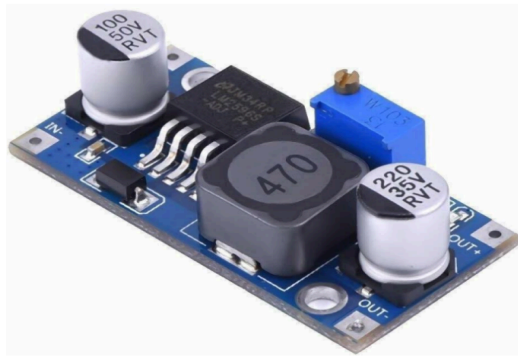


Figure 4: Buck Convertor

LM2596 Buck Converter module (Fixed 5V) is an easy-to-use, nonsynchronous, step-down DC-DC converter. The module can deliver 5V constant output with DC current load of 3A with excellent line and load regulation. If the current exceeds 2A then a heat sink is required. The recommended minimum input voltage is 9V if the current draw is higher than 1A. Input Voltage: 5V – 40V DC

## 5. COMMUNICATION PROTOCOLS

To facilitate the communication between devices and components, like microcontrollers, sensors, and other peripherals, several communication protocols have been developed for efficient and reliable exchange of data.

Among the most widely used protocols in embedded systems are UART , SPI and I2C.

### 5.1 UART (Universal Asynchronous Receiver/Transmitter)

#### Overview

UART is a hardware communication protocol that facilitates serial communication between two devices. It is a point-to-point communication system, meaning it only allows two devices to communicate directly. Unlike other protocols, UART does not require a clock signal to synchronize data transmission; instead, it uses start and stop bits to define the beginning and end of data packets.

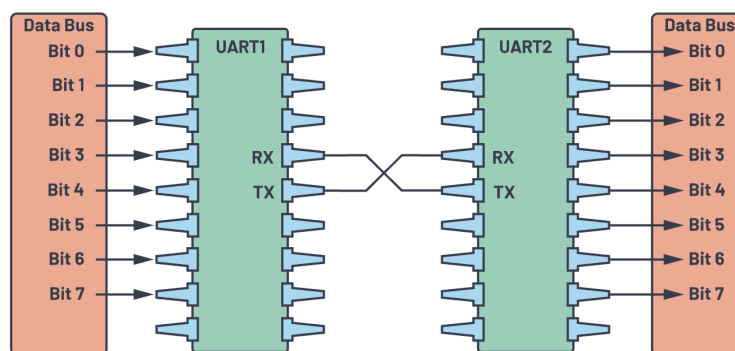


Figure 5: UART Communication

#### Architecture and Operation

- **Data Transmission:** UART transmits data serially, one bit at a time, over a single wire. Data is typically sent in 8-bit packets, but the packet size can vary depending on the configuration.
- **Full-Duplex:** UART supports simultaneous data transmission and reception
- **Baud Rate:** The speed of communication is defined by the baud rate, which specifies the number of bits transmitted per second. Both devices must operate at the same baud rate for successful communication.

- **Start and Stop Bits:** To synchronize data transmission, UART adds a start bit before each data packet and one or two stop bits after the data packet.
- **Parity Bit:** An optional parity bit can be used for error checking. It helps ensure that the data received is correct, though it cannot correct errors.

## Advantages

- **Simplicity:** UART is simple to implement and requires minimal external components.
- **No Clock Required:** Unlike synchronous protocols, UART does not require a clock signal, reducing the number of wires needed.
- Long-distance communication capability.

## Limitations

- **Limited to Two Devices:** UART can only connect two devices directly.
- **Lower Speed:** Compared to other protocols like SPI, UART is slower, typically supporting speeds up to a few Mbps.

## 5.2 SPI (Serial Peripheral Interface)

### Overview

SPI is a synchronous communication protocol used to transfer data between a master device and one or more slave devices. It is widely used in applications requiring high-speed communication and multiple slave devices, such as sensors, displays, and memory cards.

### Architecture and Operation

- **Master-Slave Configuration:** In SPI, one device is designated as the master, and the others are slaves. The master controls the communication by generating a clock signal.
- **Full-Duplex Communication:** SPI supports full-duplex communication, meaning data can be sent and received simultaneously.
- **Four-Wire System:**
  - **MOSI (Master Out Slave In):** The line through which the master sends data to the slave.
  - **MISO (Master In Slave Out):** The line through which the slave sends data to the master.
  - **SCK (Serial Clock):** The clock line, generated by the master, synchronizes data transmission.

- **SS/CS (Slave Select/Chip Select):** A line used by the master to select which slave to communicate with.

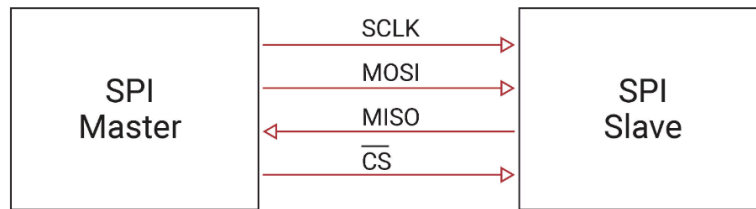


Figure 6: SPI Communication

### Advantages

- **High Speed:** SPI supports much higher data rates than UART and I2C, making it suitable for applications requiring fast data transfer.
- **Multiple Slaves:** Multiple slave devices can be connected to a single master, with each slave having its own SS line.
- **Simplicity in Protocol:** SPI is simple to implement, with minimal overhead in the data packets.

### Limitations

- **Wiring Complexity:** The need for separate SS lines for each slave increases wiring complexity.
- **No Acknowledgment:** SPI lacks a built-in acknowledgment mechanism, so the master has no way of knowing if a slave has received the data correctly.
- **No Multi-Master Support:** SPI does not support multiple masters, limiting its use in certain applications.

## 5.3 I2C (Inter-Integrated Circuit)

### Overview

I2C is a synchronous, multi-master, multi-slave communication protocol designed to connect multiple low-speed peripherals to a microcontroller or processor. It uses only two wires, making it ideal for communication between devices within a short distance, such as on a single PCB.

## Architecture and Operation

- **Two-Wire System:**
  - **SDA (Serial Data Line):** The line used for data transfer between devices.
  - **SCL (Serial Clock Line):** The clock line used to synchronize data transfer.
- **Addressing:** Each device on the I2C bus has a unique address, allowing the master to communicate with multiple devices using the same two wires.
- **Master-Slave Configuration:** In I2C, the master device initiates communication and controls the clock line. Multiple masters can be connected to the same bus, but only one can be active at a time.
- **Data Frames:** Data is transmitted in frames, starting with a start condition and ending with a stop condition. Each frame includes an address byte, followed by data bytes, and optionally, acknowledgment bits.
- **Acknowledge Mechanism:** After each byte is transmitted, the receiver sends an acknowledgment bit to confirm receipt. This ensures reliable communication.

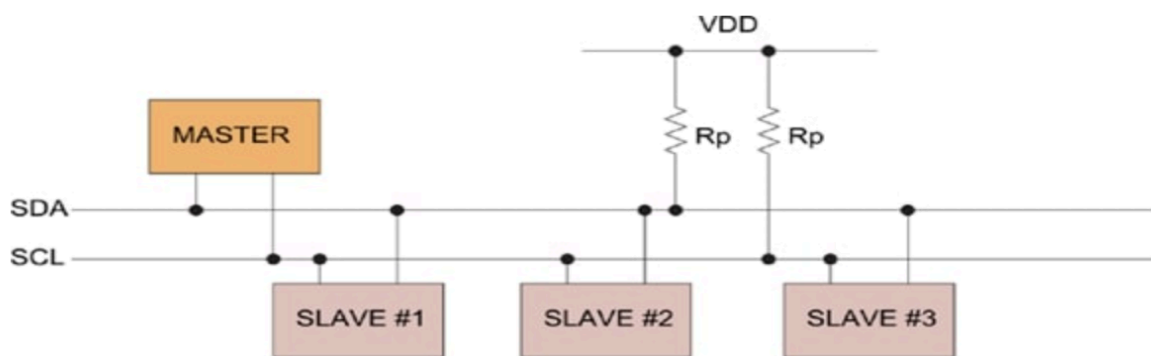


Figure 7: I2C Communication

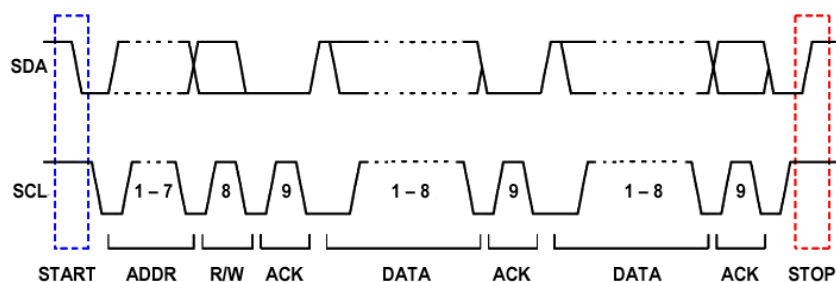


Figure 8: I2C Timing Diagram

## Advantages

- **Simplicity in Wiring:** I2C requires only two wires, regardless of the number of devices, simplifying the hardware design.
- **Multi-Master and Multi-Slave:** I2C supports multiple masters and multiple slaves, providing flexibility in system design.
- **Built-in Acknowledgment:** The acknowledgment mechanism ensures that data is received correctly, improving reliability.

## Limitations

- **Speed:** I2C is slower than SPI, typically supporting speeds up to 400 kbps (Standard Mode), 1 Mbps (Fast Mode), or 3.4 Mbps (High-Speed Mode).
- **Bus Contention:** In systems with multiple masters, bus contention can occur, requiring arbitration mechanisms to resolve conflicts.
- **Limited Distance:** I2C is designed for short-distance communication, typically within a single PCB.

## 6. YOLO (You Only Look Once)

YOLO (You Only Look Once) is a state-of-the-art object detection algorithm known for its real-time performance and high accuracy. It is used to detect and classify objects in images or videos in a single pass through a neural network. It was first introduced by Joseph Redmon and Santosh Divvala in 2016 and has evolved through several versions, each enhancing its speed and accuracy.

### How It Works:

- **Single Neural Network:** YOLO uses a single convolutional neural network (CNN) to predict bounding boxes and class probabilities from full images in one evaluation. This contrasts with traditional methods that typically involve multiple stages.
- **Grid Division:** The algorithm divides an image into an  $S \times SS \times SS \times S$  grid. Each grid cell predicts bounding boxes and probabilities for objects whose centers fall within the cell.
- **Bounding Box Predictions:** Each grid cell predicts multiple bounding boxes with associated confidence scores and class probabilities. YOLO then applies non-max suppression to eliminate duplicate boxes and refine the final detections.

### Key Features:

- **Real-Time Processing:** YOLO's architecture allows for fast object detection, making it suitable for real-time applications such as video surveillance and autonomous driving.
- **Unified Detection:** By treating detection as a single regression problem, YOLO simplifies the pipeline and speeds up computation compared to multi-stage methods.

### Applications:

- **Surveillance:** Monitoring and identifying objects in real-time video feeds.
- **Autonomous Vehicles:** Detecting pedestrians, vehicles, and other obstacles.
- **Robotics:** Real-time object recognition and interaction.
- **Augmented Reality:** Enhancing real-world environments with virtual information.

### Advantages:

- **Speed:** Capable of processing images quickly, making it ideal for time-sensitive applications.
- **Simplicity:** Integrates detection tasks into a single end-to-end model, streamlining the process.

## 6.1 Training A Model Using Google Colab

1. Connecting the Collab session to T4 GPU - changing the run type session from CPU to T4 gpu

```
!nvidia-smi
```

Tue Aug 27 17:10:21 2024

| NVIDIA-SMI 535.104.05 |          |               |                  | Driver Version: 535.104.05 |                      | CUDA Version: 12.2 |        |
|-----------------------|----------|---------------|------------------|----------------------------|----------------------|--------------------|--------|
| GPU                   | Name     | Persistence-M | Bus-Id           | Disp.A                     | Volatile Uncorr. ECC |                    |        |
| Fan                   | Temp     | Perf          | Pwr:Usage/Cap    | Memory-Usage               | GPU-Util             | Compute M.         | MIG M. |
| 0                     | Tesla T4 | Off           | 00000000:00:04.0 | Off                        | 0                    |                    |        |
| N/A                   | 61C      | P8            | 10W / 70W        | 0MiB / 15360MiB            | 0%                   | Default            | N/A    |

| Processes:                 |    |    |     |      |              |            |  |
|----------------------------|----|----|-----|------|--------------|------------|--|
| GPU                        | GI | CI | PID | Type | Process name | GPU Memory |  |
|                            | ID | ID |     |      |              | Usage      |  |
| No running processes found |    |    |     |      |              |            |  |

2. Installing Libraries

```
[ ]  
!pip install ultralytics==8.0.20  
  
from IPython import display  
display.clear_output()  
  
import ultralytics  
ultralytics.checks()
```

3. Importing YOLO

```
[ ]  
from ultralytics import YOLO  
  
from IPython.display import display, Image
```

#### 4. Uploading the custom dataset:

- i. Preparing dataset - We download the datasets from roboflow and upload them on google drive to be able to access directly on the google colab notebook.

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

[ ]
%cd /content/drive/MyDrive/DirectionDataset
```

- ii. Adding path to the Data.yaml file

```
data.yaml X
C: > Users > tanis > Downloads > DirectionDataset > data.yaml
1 train: ../train/images
2 val: ../valid/images
3 test: ../test/images
4
5 nc: 3
6 names: ['Red']
7
```

#### 5. Training the YOLO model

```
%cd /content/drive/MyDrive/DirectionDataset

!yolo task=detect mode=train model=yolov10s.pt data= data.yaml epochs=25 imgsz=224 plots=True
```

6. Extracting best.pt file - The best.pt and last.pt files are the outputs of the training process. After running the training command, these files are saved in the default output directory (often named runs/train/exp or similar) and contain:
  - best.pt: The model checkpoint with the best performance on the validation set.
  - last.pt: The model checkpoint from the final epoch of training.

We can find these files by navigating to the output directory specified or checking the logs for the exact path.

## 6.2 Algorithm for Object Detection and Motor Control Using YOLO, OpenCV, and PySerial

### Initialize Environment:

- Importing necessary libraries: cv2 for OpenCV, supervision for annotating the bounding boxes, YOLO from ultralytics for object detection, and Serial for Arduino communication, torch(CUDA) for GPU acceleration, and other utility libraries like math and time.
- Load the pre-trained YOLO model for object detection (.pt file).
- Initialize the webcam to capture live video feed.

### Function to Get User Input for Arduino Port (get\_user\_port):

This function lists all available COM ports on the machine and prompts the user to select one to communicate with an Arduino. The port is returned as a string for use in serial communication.

### Main Function:

- **Device Selection:** It checks if CUDA (GPU) is available using torch.cuda.is\_available(). If it is, it sets the device to 'cuda', otherwise, it uses the CPU. This enables the program to leverage the GPU for faster object detection if available.
- **Serial Communication Setup:** It opens the serial port (selected by the user earlier) to establish communication with an Arduino at a baud rate of 9600.
- **YOLO Model Loading:** The YOLO model is loaded using YOLO("./ppe.pt"), where ppe.pt is a trained YOLO model file. This model is transferred to the selected device (GPU or CPU) for inference.
- **Video Capture Setup:** The code uses cv.VideoCapture(0) to capture video input from the default camera (device 0).
- **Annotators Setup:** Two annotators are initialized for displaying bounding boxes and labels on the detected objects using the supervision library.
- **Target Class:** The target class is specified as "P". The YOLO model will detect this specific class (likely related to the dataset used for training, like detecting "Purple ball").

### Main Loop for Video Frame Processing:

- The program continuously captures video frames in a loop until the user quits (q key press).

- **Frame Capture:** A frame is captured from the video stream using `cap.read()`.
- **Object Detection:** The frame is passed to the YOLO model for inference using `model(frame, stream=True)`. The model returns the detected bounding boxes and their corresponding classes.
- **Bounding Box Processing:**
  - For each bounding box in the frame, the code checks the confidence score (`box.conf[0]`) and compares it with a threshold ( $> 0.5$ ).
  - It also checks if the detected class corresponds to the target class (e.g., "P").
  - The bounding box coordinates (X1, Y1, X2, Y2) are extracted, and the box is drawn on the frame using the supervision annotators (`bounding_box_annotator` and `label_annotator`).
  - The area of the bounding box is calculated as  $(X2 - X1) * (Y2 - Y1)$ .

### **Center and Area Calculation:**

- The program calculates the center of the bounding box as (`center_xp`, `center_yp`).
- The approximate area of the frame minus a margin (100 pixels on each side) is computed for later comparison.

### **Threshold-Based Data Calculation:**

- Based on the position of the object's center (`center_xp`), the frame is divided into sections, and corresponding commands (A, B, C, D, f, or S) are generated.
  - **A/B/D/C:** Commands are based on whether the object is to the left or right of the frame.
  - **f:** Indicates that the object is roughly in the center.
  - **S:** Stop command if no significant detection or if the center doesn't meet any condition.

### **Communication with Arduino:**

- If the generated command differs from the previous one, the data is sent via serial communication to the Arduino.
- The code reads a response from the Arduino (`serialData`) and prints it.

### **Exit Condition:**

- The program continuously processes frames until the user presses the 'q' key to quit the program.

**Exception Handling:**

- The program has error handling for serial communication issues (SerialException) and other potential errors (Exception).

**Closing Resources:**

- Finally, the serial port is closed, and all resources are released when the program ends.

## 7. CODE FOR OBJECT DETECTION

```
import cv2 as cv
import supervision as sv
from ultralytics import YOLO
import serial
from serial.tools import list_ports
import math
import torch

def get_user_port():
    ports = list_ports.comports()
    ports_list = []

    for one in ports:
        ports_list.append(str(one))
        print(str(one))

    com = input("Select Com Port for Arduino (#): ")
    return str(com)

def main():
    ser = None
    try:
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        print(device)
        port = get_user_port()
        ser = serial.Serial(port, 9600, timeout=1) # specifying baud rate
        model = YOLO("./ppe.pt") #path to the .pt file (trained model)
        model.to(device)

        cap = cv.VideoCapture(0)
        # Box annotations
        bounding_box_annotator = sv.BoxAnnotator()
        label_annotator = sv.LabelAnnotator()
        target_class = "P"
        previous_data = ""

        while True:
            ret, frame = cap.read()
            if not ret:
                print("Unable to capture Video")
                break
```

```

# Bounding Box processing
results = model(frame, stream=True)
for result in results:
    boxes = result.boxes
    X1 = Y1 = X2 = Y2 = 0
    area = 0
    for box in boxes:
        if math.ceil(box.conf[0]) > 0.5 and model.names[int(box.cls[0])] == target_class:
            X1, Y1, X2, Y2 = box.xyxy[0]
            X1, Y1, X2, Y2 = [int(X1), int(Y1), int(X2), int(Y2)]
            detections = sv.Detections.from_ultralytics(result)
            annotated_image = bounding_box_annotator.annotate(scene=frame, detections=detections)
            annotated_image = label_annotator.annotate(scene=annotated_image,
detections=detections)
            cv.imshow("Video", annotated_image)
            area = int((X2 - X1) * (Y2 - Y1))

# Communication with Arduino
center_xp = (X2 + X1) // 2
center_yp = (Y2 + Y1) // 2
approx_framearea = (frame.shape[1] - 100) * (frame.shape[0] - 100)
threshold = frame.shape[1] // 5
data = ""
print("Threshold: ", threshold)
if center_xp < threshold:
    data = "A"
elif center_xp < 2 * threshold:
    data = "B"
elif center_xp > 4 * threshold:
    data = "D"
elif center_xp > 3 * threshold:
    data = "C"
elif ((center_xp > 2 * threshold) and (center_xp < 3 * threshold)):
    data = "F"

else:
    data = "S"

if data != previous_data:
    ser.write(data.encode())
    serialData = ser.readline().decode("utf-8").strip()
    print("the data is: ", serialData)
    previous_data = data

```

```
        if cv.waitKey(1) == ord('q'):
            break

    except serial.SerialException as e:
        print(f"Serial communication error: {e}")

    except Exception as e:
        print(f"Error: {e}")

    finally:
        if ser is not None and ser.is_open:
            ser.close()
            print("Serial port closed.")

if __name__ == "__main__":
    main()
```

## 8. CODE FOR ARDUINO AND MOTOR CONTROL

```
int PWM1= 2;
int PWM2= 5;
int SLEEP1=3;
int SLEEP2=7;
int DIR1=4;
int DIR2=6;
void setup() {
  pinMode(PWM1,OUTPUT);
  pinMode(PWM2,OUTPUT);
  pinMode(SLEEP1,OUTPUT);
  pinMode(SLEEP2,OUTPUT);
  pinMode(DIR1,OUTPUT);
  pinMode(DIR2,OUTPUT);
  digitalWrite(SLEEP1,LOW);
  digitalWrite(SLEEP2,LOW);
  Serial.begin(9600);
  Serial.setTimeout(1);
}
```

//let us assume 1 HIGH while set a DIR such that motor will move in forward direction an LOW will result in backward direction

//1

```
void forward(int speed){
  digitalWrite(DIR1,HIGH);
  digitalWrite(DIR2,HIGH);
  analogWrite(PWM1,speed);
  analogWrite(PWM2,speed);
}
```

//2

```
void right(int speed){
  digitalWrite(DIR1,HIGH);
  digitalWrite(DIR2,HIGH);
  analogWrite(PWM1,0);
  analogWrite(PWM2,speed);
}
```

//3

```
void left(int speed){
  digitalWrite(DIR1,HIGH);
  digitalWrite(DIR2,HIGH);
  analogWrite(PWM1,speed);
  analogWrite(PWM2,0);
}
```

```

}
//4
void stop(){
    digitalWrite(DIR1,HIGH);
    digitalWrite(DIR2,HIGH);
    analogWrite(PWM1,0);
    analogWrite(PWM2,0);
}
void loop() {
    //here we will write the code for communication with python
    if(Serial.available()>0){
        String data = Serial.readString();
        Serial.println(data);
        data.trim();
        if(data=="A"){left(100);}
        if(data=="B"){left(75);}
        if(data=="C"){right(75);}
        if(data=="D"){right(100);}
        if(data=="S"){stop();}
        if(data=="F"){forward(80);}
        if(data=="F"){forward(150);}
    }
    delay(50);
}

```

## 9. PYSERIAL

PySerial is a Python library which is capable of communicating with external hardware by facilitating serial communication. It provides access to the serial port on a variety of operating systems. It is widely used for communication between microcontrollers and computers, enabling the exchange of data over serial communication protocols.

To establish communication between arduino and python, we connect arduino to the port, and for that we have to be aware about the details regarding the available port and COM to which arduino is connected. For this, we generate the list of ports available using python.

```
import serial.tools.list_ports
```

```
"""  
this statement is used in Python to access  
functionalities for listing and working with  
serial ports  
"""
```

```
ports = serial.tools.list_ports.comports()
```

```
"""  
comport() is a function which is used to list  
all available serial ports on a computer.  
this command is storing the list of COMs in ports variable."""
```

```
portList = []
```

```
#this command initialize an empty list to store port information
```

```
for onePort in ports:
```

```
"""  
onePort:- It represents an individual SerialPort object from the  
list returned by comports(). Each SerialPort object contains information about a specific serial port,  
such as its name, description, and hardware ID.  
"""
```

```
portList.append(str(onePort))
```

```
"""  
It converts the onePort object to its string representation and adds it to the portList list which we have  
initiated above. append() is a pre-defined method used with lists to add a single item to the end of the list
```

```
'''
print(str(onePort))
#this command print list
```

Now as we know, to which COM port our Arduino is connected, We will write code which will transfer data from arduino to python or python to arduino : -

### i) For Python : -

```
import serial
# Initialize serial connection

ser = serial.Serial('COM7', baudrate = 9600, timeout = 0.1)
# This opens the serial port COM7 with a baud rate of 9600 and a timeout of 0.1 seconds.

msgWR = input('Write message to Arduino: ')
arduino.write(bytes(msgWR, 'utf-8'))

# This prompts the user to enter a message, which is stored in variable msgWR, which is then sent to the
# Arduino as UTF-8 encoded bytes.

msgRD = arduino.readline()
if msgRD:
    msgRD = msgRD.decode('utf-8').strip()
    print(f'Message from Arduino: {msgRD}')
else:
    print('No message received from Arduino.')
'''
```

The code reads a line from the Arduino and decodes it from UTF-8. It also uses strip() to remove any leading or trailing whitespace and newlines. If no data is read (i.e., msgRD is empty), it prints a message indicating no data was received.

### ii) For Arduino IDE

If we have to read data from python then : -

```
void setup() {
    Serial.begin(9600);
    Serial.setTimeout(1);
}
void loop() {
    string message;
    while(!Serial.available()) {
        message = Serial.readString();
    }
```

If we want to send data to python, just print it on a serial monitor using `Serial.print(arduinoData);` , here `arduinoData` is the data which you want to send to python, python will read the message.

**Important points :-**

- Close your Serial monitor before running your python code or else you would end up getting an “open port permission error”.
- Your baud rate should be the same in both python and Arduino IDE.
- Timeout Configuration: The timeout parameter defines how long the `readline()` method waits for data before giving up. If you find that data isn't being read properly, consider increasing the timeout.

### 10. CIRCUIT DIAGRAM:

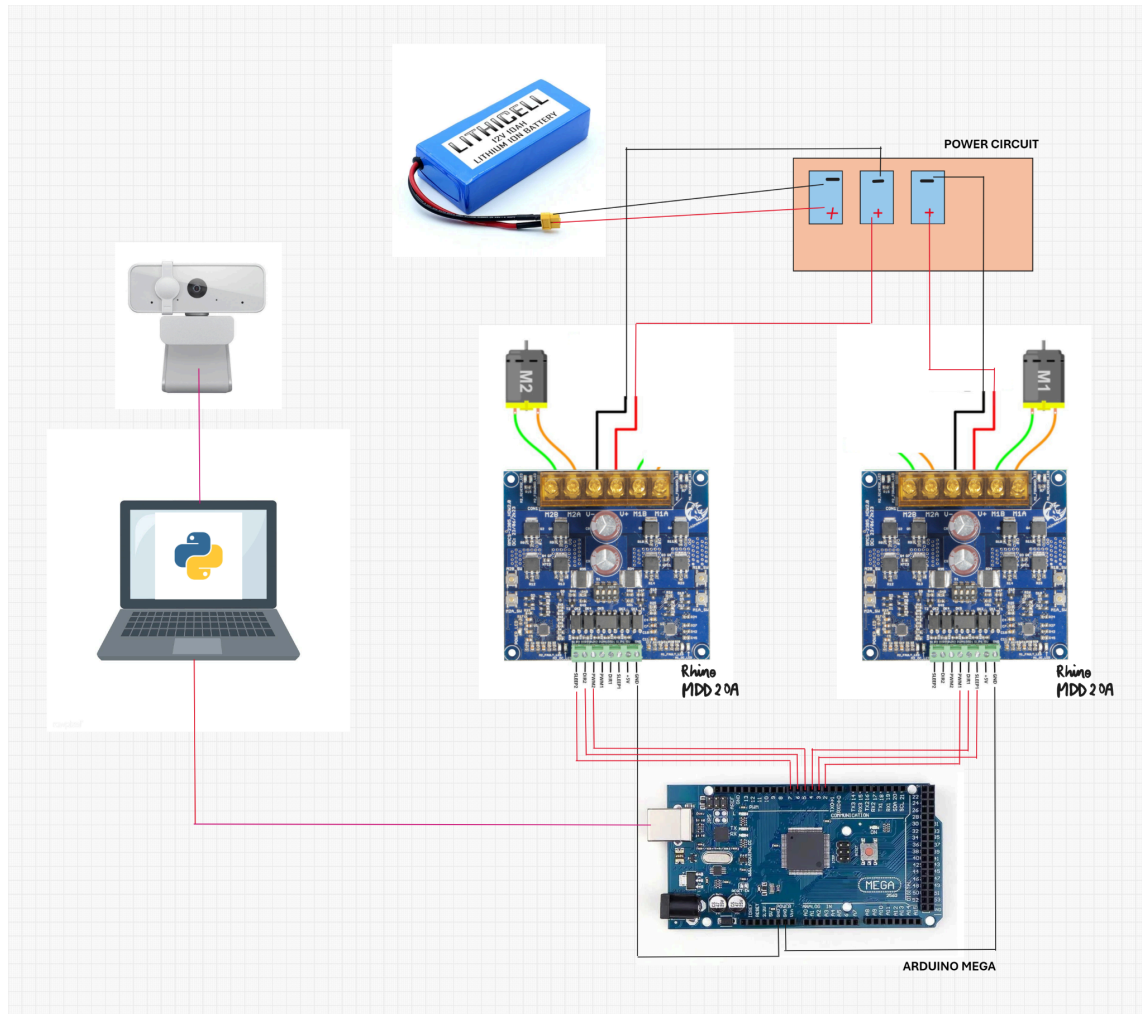
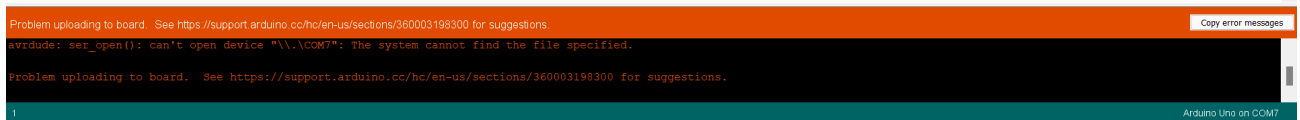


Figure 10: Circuit Diagram

## 11. ERRORS AND THEIR SOLUTIONS

### Error1: Port not found

Solution: Go to Tools>Port>Select Arduino Mega



### Error2: “fbgemm.dll” or one of its dependencies are not found

Solution: This error occurs due to missing dependency file of ultralytics that is fbgemm.dll which is a library used for matrix multiplication. Install fbgemm.dll from google and paste the file in system32 of windows OR install **Microsoft Visual C/C++** tool which contains the required runtime component. If that doesn't work uninstall and reinstall PyTorch



### Error 3: Classnames of bounding boxes formed are not accessed

Solution: While accessing classnames we have used result(i.e. generator object formed by YOLO model)

And fetched the class name by box.cls property which returns the corresponding id but the class.names method was unable to fetch the class name from class id. So we directly used the model to fetch the class name by giving class id as argument.

### Error 4: Cuda not recognized by the PyTorch model

Solution : This error occurs as the cuda toolkit downloaded in our device is not configured to the version of the cuda gpu of our device. Downloading torch, torchvision, torchaudio, Cuda toolkit and tensorflow of the version compatible with the GPU(12.6 in our case) resolves the error.



```
cuda.py
1 import torch
2 if(torch.cuda.is_available()):
3     print("cuda device was found")
4 else:
5     print("No cuda device recognized")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
PS D:\B.Tech\Coding\Python\Final File of Code> python -U "d:\B.Tech\Coding\Python\Final File of Code\cuda.py"
No cuda device recognized
PS D:\B.Tech\Coding\Python\Final File of Code> 
```

### Error 5: Conflict of commands happening between python and arduino

Solution: We have used GPU for frame rate acceleration but due to fast processing of image, arduino received multiple commands due to which a conflict of commands took place while reading data in arduino. So we have conditionally restricted sending the same command multiple times to arduino which resolved the issue.

## 12. REFERENCES

- Key Machine Learning Terminologies  
<https://www.youtube.com/watch?v=ApNgG-w3UXI>
- Neural Networks Explained  
[https://www.youtube.com/watch?v=ER2It2mIagI&list=PLq6bkmWUX0N9tdQvwvml5\\_QE9SshWQ7hj&index=7](https://www.youtube.com/watch?v=ER2It2mIagI&list=PLq6bkmWUX0N9tdQvwvml5_QE9SshWQ7hj&index=7)
- Arduino Mega Documentation  
<https://docs.arduino.cc/hardware/mega-2560/>
- Object detection (Open CV)  
[https://www.youtube.com/watch?v=WgPbbWmnXJ8\]\(https://www.youtube.com/watch?v=WgPbbWmnXJ8](https://www.youtube.com/watch?v=WgPbbWmnXJ8](https://www.youtube.com/watch?v=WgPbbWmnXJ8)
- Training a custom dataset YOLO  
[https://www.youtube.com/watch?v=29tnSxhB3CY\]\(https://www.youtube.com/watch?v=29tnSxhB3CY](https://www.youtube.com/watch?v=29tnSxhB3CY](https://www.youtube.com/watch?v=29tnSxhB3CY)
- Open CV - Full tutorial with Python  
[https://www.youtube.com/watch?v=oXlwWbU8l2o\]\(https://www.youtube.com/watch?v=oXlwWbU8l2o](https://www.youtube.com/watch?v=oXlwWbU8l2o](https://www.youtube.com/watch?v=oXlwWbU8l2o)
- Complete Python tutorial  
[https://www.youtube.com/watch?v=e\\_uQrJ0TkZlc\]\(https://www.youtube.com/watch?v=e\\_uQrJ0TkZlc](https://www.youtube.com/watch?v=e_uQrJ0TkZlc](https://www.youtube.com/watch?v=e_uQrJ0TkZlc)
- Pyserial guide  
[https://pyserial.readthedocs.io/en/latest/pyserial\\_api.html#\]\(https://pyserial.readthedocs.io/en/latest/pyserial\\_api.html](https://pyserial.readthedocs.io/en/latest/pyserial_api.html#](https://pyserial.readthedocs.io/en/latest/pyserial_api.html)
- Cuda C++ programming guide  
[https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#c-language-extensions\]\(https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#c-language-extensions](https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#c-language-extensions](https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#c-language-extensions)